گزارش پروژه‌ی درس رایانش ابری

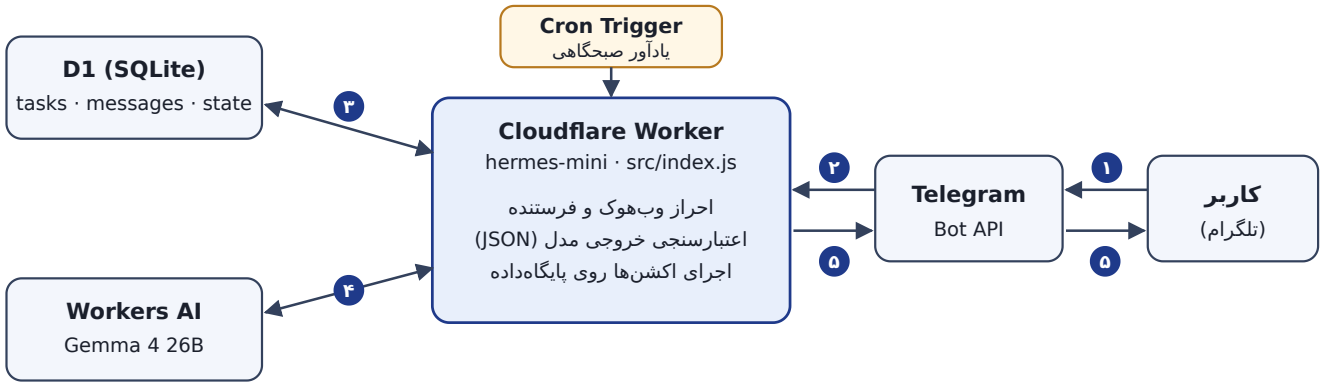
ربات تلگرام سرورلس «یادم نره» روی Cloudflare

نام و نام خانوادگی	امین خیرآبادی
نام ربات	یادم نره (نام پروژه و Worker: hermes-mini)
آدرس زنده	https://hermes-mini.amin-kheirabadi4182.workers.dev
تاریخ	۱۳ آبان ۱۴۰۵

۱. خلاصه

«یادم نره» یک ربات تلگرام است که کارهای روزانه را از متن آزاد فارسی ثبت، تیک و حذف می‌کند. هیچ دستور اسلشی وجود ندارد؛ مدل زبانی هر پیام را به یکی از شش اکشن ثابت (`add`، `done`، `undone`، `del`، `list`، `chat`) تبدیل می‌کند. کل سیستم روی پلن رایگان Cloudflare اجرا می‌شود: Workers برای محاسبه، D1 برای پایگاه‌داده و Workers AI برای مدل زبانی، بدون هیچ سرور شخصی.

۲. معماری نهایی



۱. کاربر پیام فارسی می‌فرستد.

۲. تلگرام آن را با `POST /webhook` به Worker می‌فرستد و هدر `X-Telegram-Bot-Api-Secret-Token` را همراه آن می‌گذارد. Worker هدر را بررسی می‌کند (نامعتبر ← ۴۰۳)، فرستنده را با فهرست مجاز تطبیق می‌دهد و `update_id` را برای جلوگیری از پردازش تکراری ثبت می‌کند.

۳. لیست شماره‌دار کارها، «آخرین کار لمس‌شده» و چند پیام آخر از D1 خوانده می‌شود و پس از اجرای اکشن، تغییرات در D1 نوشته می‌شود.

۴. این اطلاعات به Workers AI داده می‌شود و مدل فقط یک خط JSON برمی‌گرداند؛ Worker آن را اعتبارسنجی می‌کند.

۵. پاسخ فارسی با `sendMessage` به کاربر می‌رسد. در همه‌ی حالت‌ها به تلگرام کد ۲۰۰ برمی‌گردد تا همان پیام دوباره تحویل نشود. Cron Trigger هم مستقل از پیام کاربر Worker را برای یادآور صبحگاهی بیدار می‌کند (بونوس).

۳. تصمیم‌های طراحی (فاز ۱)

پرسش	تصمیم	دلیل
وب‌هوک یا polling؟	وب‌هوک	Worker فقط هنگام رسیدن رویداد اجرا می‌شود و فرایند همیشه‌روشن ندارد. polling یا به یک حلقه‌ی دائمی نیاز دارد یا با Cron (حداقل هر یک دقیقه) تأخیر و درخواست‌های بیهوده ایجاد می‌کند.
چرا FaaS؟	Cloudflare Workers	بار کار کوچک و پراکنده است (چند پیام در روز). هزینه فقط به‌ازای اجراست، نیازی به مدیریت سرور و سیستم‌عامل نیست و در پلن رایگان می‌گنجد.
چرا D1 و نه KV؟	D1 (SQLite)	داده رابطه‌ای است و باید بر اساس فیلد پرس‌وجو شود (کارهای باز، انجام‌شده‌ی امروز). در D1 دستورهای `batch` تراکنشی‌اند و قید `CHECK` و ایندکس داریم. KV فقط کلید-مقدار است، در نهایت سازگار (eventually consistent) است و سقف نوشتن رایگانش ۱٬۰۰۰ بار در روز است؛ هر پیام حدود ۱۰ نوشتن دارد.

وضعیت (state) کجاست؟	D1	Worker بدون حالت است. جدول‌ها: `tasks` ، `messages` (حافظه‌ی مکالمه)، `state` (آخرین کار لمس‌شده، تأیید حذف، آخرین یادآور) و `processed_updates` . Secretها در secret store و تنظیمات غیرحساس در `wrangler.toml` هستند.

۴. وب‌هوک امن و احراز هویت (فاز ۲)

- هدر secret با مقایسه‌ی زمان‌ثابت بررسی می‌شود؛ نبود یا نادرستی آن ← **۴۰۳**. اگر secret روی سرور تنظیم نشده باشد هم همه‌ی درخواست‌ها رد می‌شوند.
- فقط شناسه‌ی عددی (یا شناسه‌های) مجاز تعریف‌شده در Secret پذیرفته می‌شود و فقط در چت خصوصی. فرستنده‌ی غیرمجاز **بی‌صدا** نادیده گرفته می‌شود (کد ۲۰۰، بدون پاسخ، فقط یک لاگ بدون متن پیام).
- نقطه‌ی `GET /health` وضعیت Worker و اتصال D1 را نشان می‌دهد (۲۰۰ یا ۵۰۳) و روی آدرس زنده در دسترس است.
- `BOT_TOKEN` ، `WEBHOOK_SECRET` و `ALLOWED_USER_ID` فقط با `wrangler secret put` ثبت شده‌اند و در کد یا مخزن نیستند. متغیرهای غیرحساس (نام مدل، ساعت یادآور) در بخش `[vars]` هستند.

۵. پایگاه‌داده و تعریف «امروز» (فاز ۳)

جدول `tasks` شامل `id` ، `title` ، `status` (فقط `open` یا `done` با قید `CHECK`)، `created_at` ، `created_day` ، `done_at` و `done_day` است. اسکیما در فایل جداگانه‌ی `migrations/0001_init.sql` نگهداری و با `wrangler d1 migrations apply` اعمال می‌شود، نه از داخل کد.

تعریف «امروز»: روز تقویمی به وقت تهران (`Asia/Tehran` ، یعنی UTC+3:30؛ ایران از سال ۱۴۰۱ ساعت تابستانی ندارد). Worker منطقه‌ی زمانی ندارد و روی UTC اجرا می‌شود، پس تاریخ تهران با `Intl.DateTimeFormat` محاسبه می‌شود. زمان‌ها به UTC ذخیره می‌شوند و «روز تهران» جداگانه در `created_day` و `done_day` نگه‌داری می‌شود. نیمه‌شب تهران برابر ساعت ۲۰:۳۰ به وقت UTC است و در همان لحظه روز عوض می‌شود. «انجام‌شده‌ی امروز» یعنی `done_day` برابر امروز تهران.

۶. لایه‌ی هوش مصنوعی (فاز ۴)

- **مدل:** Gemma 4 26B A4B روی Workers AI با دمای صفر. اگر خطا بدهد، مدل جایگزین GLM-4.7-Flash فراخوانی می‌شود.
- **ورودی مدل:** لیست شماره‌دار کارها (باز و انجام‌شده‌ی امروز)، «آخرین کار لمس‌شده»، ۱۰ پیام آخر (حافظه‌ی مکالمه) و تاریخ تهران. مدل فقط **شماره** برمی‌گرداند و تبدیل شماره به رکورد، کار خود کد است.

خروجی مدل (یک خط JSON)	کار
`{"action":"add","items":["..."]}`	افزودن یک یا چند کار
`{"action":"done","targets":[1]}` یا `{"all":true}`	انجام‌شد (و `undone` برای برگرداندن به حالت باز)
`{"action":"del","targets":[2]}` یا `{"all":true}`	حذف
`{"action":"list","scope":"open|today|all"}`	نمایش لیست
`{"action":"chat","reply":"..."}`	گفتگو یا پرسیدن سؤال شفاف‌کننده

- **اعتبارسنجی:** اولین JSON متعادل از خروجی مدل استخراج می‌شود (حتی اگر دورش متن یا ``` باشد) و با اسکیمای خودمان سنجیده می‌شود: اکشن مجاز، نوع و دامنه‌ی فیلدها، و فقط فیلدهای مجاز عبور می‌کنند. شکست ← یک بار تلاش مجدد با یادآوری ← **مسیر امن**: هیچ چیزی در D1 نوشته نمی‌شود و ربات از کاربر می‌خواهد دوباره بنویسد. خطای مدل یا گذشتن ۱۵ ثانیه هم همین مسیر را دارد.
- **موارد خاص و مبهم:** شماره‌ی خارج از لیست («شماره ۹ را حذف کن») ← پیام خطا همراه لیست. مرجع نامشخص («اون یکی را پاک کن») ← مدل سؤال شفاف‌کننده می‌پرسد و حدس نمی‌زند. حذف گروهی («همه را حذف کن») ← ابتدا فهرست را نشان می‌دهد و تأیید «بله/نه» می‌گیرد (بدون LLM). متن داخل عنوان کارها «داده» است و دستور محسوب نمی‌شود.

۷. عملیات و استقرار (فاز ۵)

استقرار با `wrangler deploy` انجام شده و bindingهای `DB` و `AI` در `wrangler.toml` تعریف شده‌اند. Observability در همان فایل فعال است. لاگ‌ها JSON ساختاریافته‌اند و هرگز متن پیام یا توکن در آن‌ها نیست؛ لاگ زنده با `wrangler tail` دیده می‌شود.

وب‌هوک با `setWebhook` و پارامترهای `secret_token` و `max_connections=1` (تحویل پشت‌سرهم پیام‌ها) ثبت شده است. کد، migration و تنظیمات در یک مخزن Git هستند و هیچ توکنی در آن نیست (فقط فایل نمونه‌ی `.dev.vars.example`).

۸. بونوس: یادآور صبحگاهی

Cron Trigger با عبارت `30 * * * *` تنظیم شده است: دقیقه‌ی ۳۰ هر ساعت UTC برابر دقیقه‌ی ۰۰ به وقت تهران است. کد بررسی می‌کند که ساعت تهران برابر `REMINDER_HOUR` (پیش‌فرض ۸) باشد و آن روز یادآور قبلاً ارسال نشده باشد؛ آنگاه لیست کارهای باز به کاربر فرستاده می‌شود.

۹. تحلیل هزینه در دو مقیاس

فرض‌ها (برآورد، نه اندازه‌گیری): یک کاربر ۳۰ پیام در روز؛ در مقیاس بزرگ هر کاربر ۲۰ پیام در روز (کل ۶ میلیون پیام در ماه). هر پیام: یک درخواست Worker، حدود ۱۰ ردیف نوشتن و ۵۰ ردیف خواندن در D1، یک فراخوانی مدل با حدود ۱٬۵۰۰ توکن ورودی و ۱۵۰ توکن خروجی، و حدود ۱۰ میلی‌ثانیه CPU. نرخ‌ها از مستندات رسمی Cloudflare (آبان ۱۴۰۵) گرفته شده است.

منبع	۱ کاربر (پلن رایگان)	۱۰٬۰۰۰ کاربر (پلن Paid)
Workers	حدود ۵۴ درخواست در روز (۳۰ پیام و ۲۴ اجرای cron) از سقف ۱۰۰٬۰۰۰ ← ۰ دلار	۶ میلیون درخواست در ماه، داخل ۱۰ میلیون اولیه ← ۵ دلار پایه + حدود ۰/۶۰ دلار CPU
D1 نوشتن	حدود ۳۰۰ ردیف در روز از سقف ۱۰۰٬۰۰۰ ← ۰ دلار	۶۰ میلیون ردیف در ماه؛ ۵۰ میلیون شامل است، ۱۰ میلیون اضافه ← حدود ۱۰ دلار
D1 خواندن	حدود ۱٬۵۰۰ ردیف در روز از سقف ۵ میلیون ← ۰ دلار	۳۰۰ میلیون در ماه از ۲۵ میلیارد شامل ← ۰ دلار
Workers AI	حدود ۵۳۰ نورون در روز از سقف رایگان ۱۰٬۰۰۰ ← ۰ دلار	حدود ۱۰۶ میلیون نورون در ماه ← حدود ۱٫۱۶۷ دلار
جمع ماهانه	**۰ دلار**	**حدود ۱٫۱۸۳ دلار (حدود ۰/۱۲ دلار برای هر کاربر)**

نرخ‌های استفاده‌شده: Workers Paid ماهانه ۵ دلار با ۱۰ میلیون درخواست و ۳۰ میلیون میلی‌ثانیه CPU، اضافه‌ی هر میلیون درخواست ۰/۳۰ دلار و هر میلیون میلی‌ثانیه ۰/۰۲ دلار؛ D1 در پلن Paid با ۵۰ میلیون نوشتن شامل و هر میلیون اضافه ۱ دلار؛ Workers AI به ازای هر ۱٬۰۰۰ نورون ۰/۰۱۱ دلار، و برای Gemma 4 26B برابر ۹٬۰۹۱ نورون به ازای هر میلیون توکن ورودی و ۲۷٬۲۷۳ نورون به ازای هر میلیون توکن خروجی.

- حدود ۹۹٪ هزینه در مقیاس بزرگ مربوط به مدل زبانی است و بیشترین سهم آن توکن ورودی (طول prompt) است؛ کوتاه‌کردن prompt یا مدل کوچک‌تر هزینه را متناسب کم می‌کند.
- در مقیاس ۱۰٬۰۰۰ کاربر طراحی هم باید تغییر کند: افزودن ستون `user_id` به جدول‌ها؛ بالا بردن `max_connections` با حفظ ترتیب پیام‌های هر کاربر؛ و ارسال یادآورها به‌صورت دسته‌ای یا با صف، چون هر اجرای Worker سقف زیرفراخوانی (subrequest) دارد و تلگرام نرخ ارسال را محدود می‌کند.
- پلن Paid فقط برای همین تحلیل فرضی است؛ خود پروژه با یک کاربر کاملاً در پلن رایگان اجرا می‌شود.